

Data Structures in C for Digital Marketing Tools

High-performance solutions for Ad Tech, SEO, and analytics

Why C in Modern Marketing?

Big Data Reality

Millions of clicks, impressions, user logs processed daily across platforms

Millisecond Latency

Real-time bidding demands sub-100ms decisions on ad serving

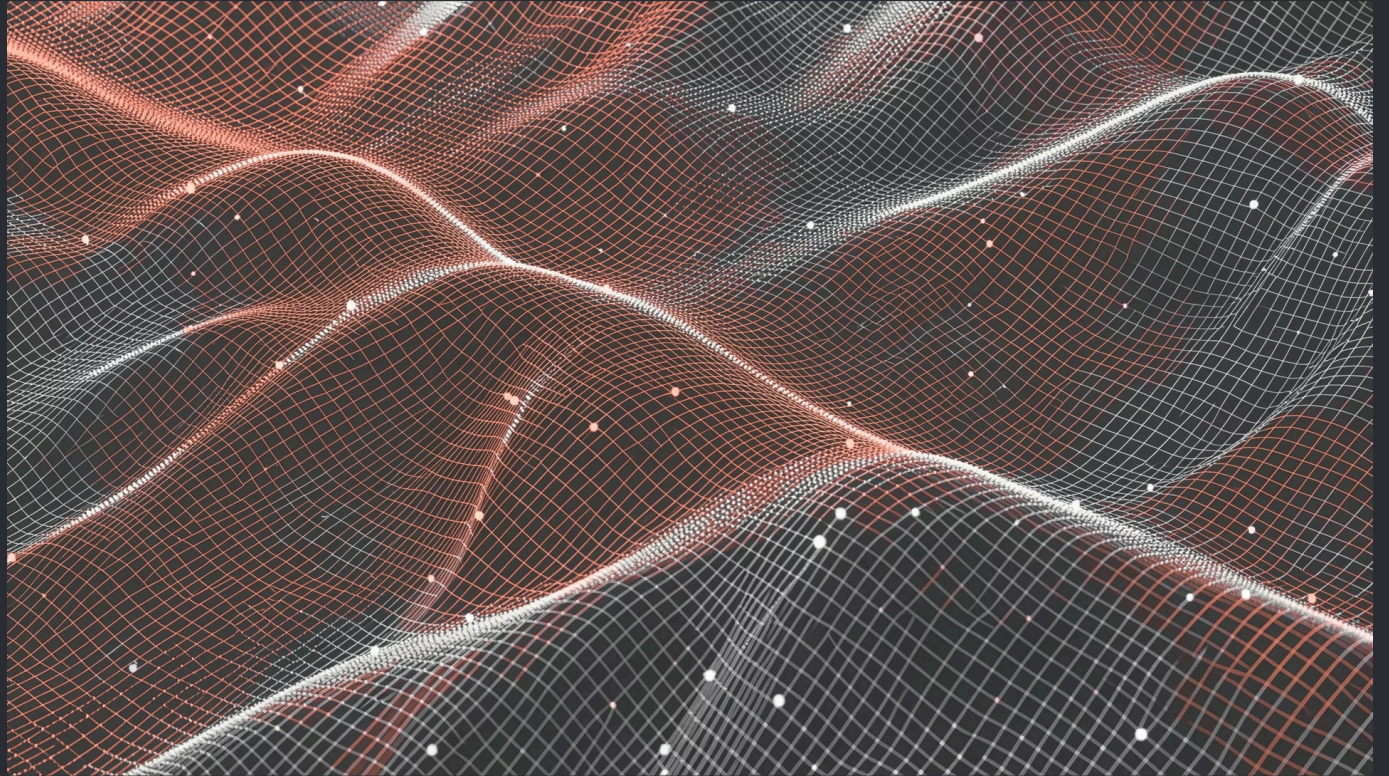
Core Engine Power

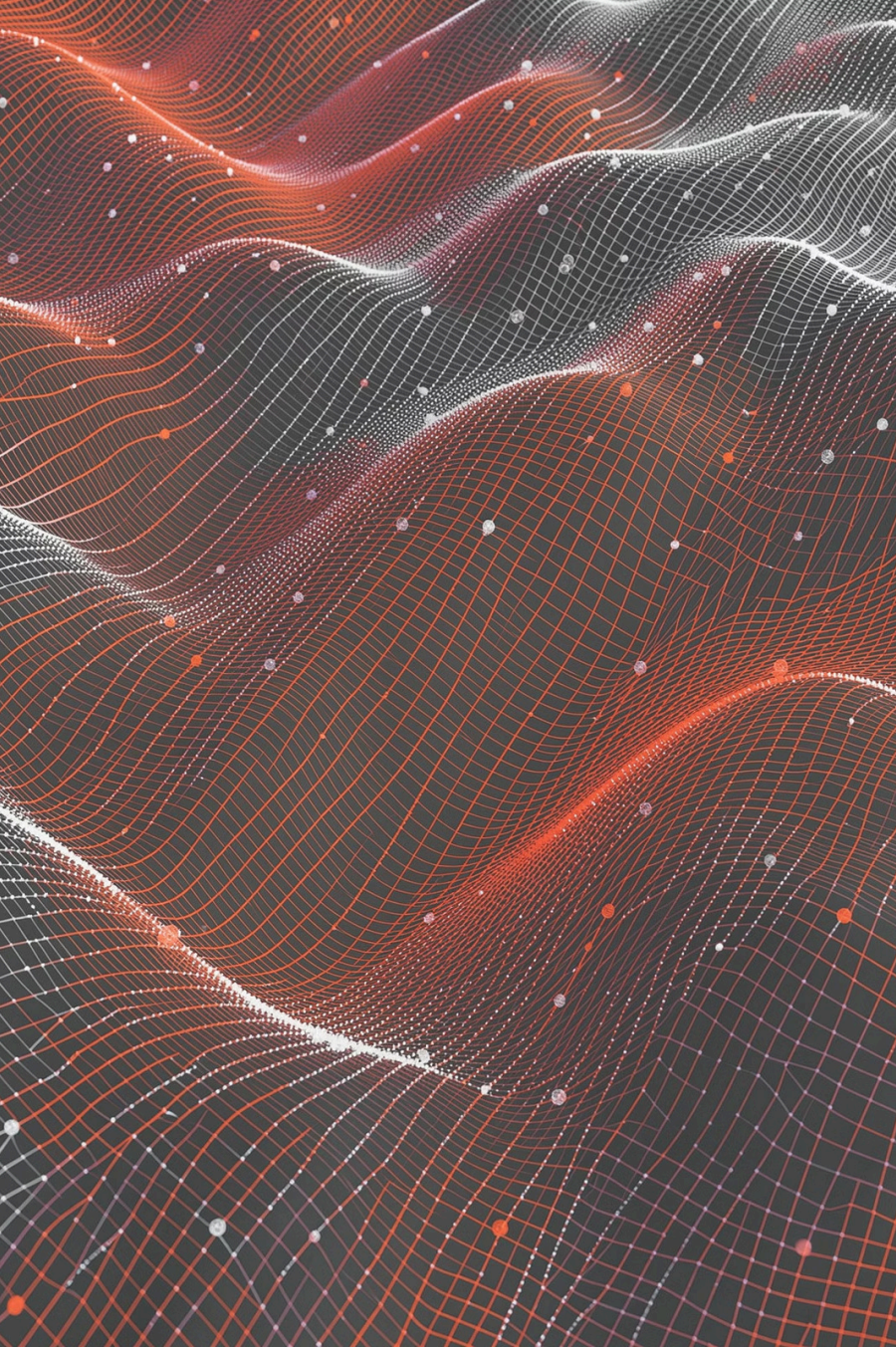
C drives ad servers, databases, routers—Python/R handle analysis only

The Core Challenge

Data Volume vs. Processing Speed

- 100,000+ requests per second
- Dynamic keywords, bids, user IDs
- Limited RAM, no memory leaks





Arrays & Strings: The Foundation

Keyword Management

Storing negative keywords, excluded IPs, ad creatives for rotation

Contiguous Memory

$O(1)$ access time enables fast iteration through millions of entries

Network Buffering

Essential for buffering incoming user data packets at scale

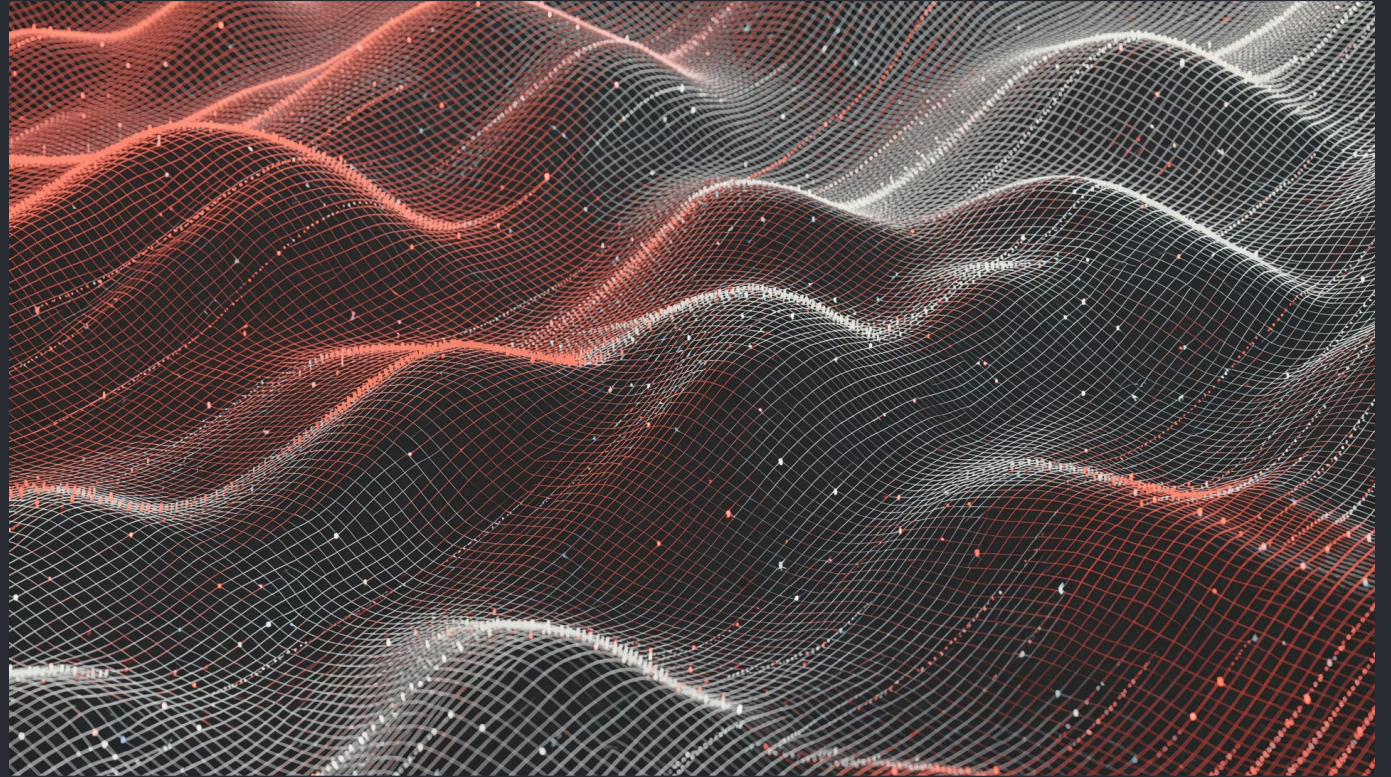
Hash Tables: Real-Time Bidding Key

User Cookies & Session Management

Instant lookup of user IDs across millions of records

Frequency Capping

Track ad view counts: UserID $\rightarrow$ View Count mapping



Why C? Average $O(1)$ complexity for search, insert, delete—critical for sub-100ms ad server decisions

Linked Lists: Dynamic Log Handling



Home



Product

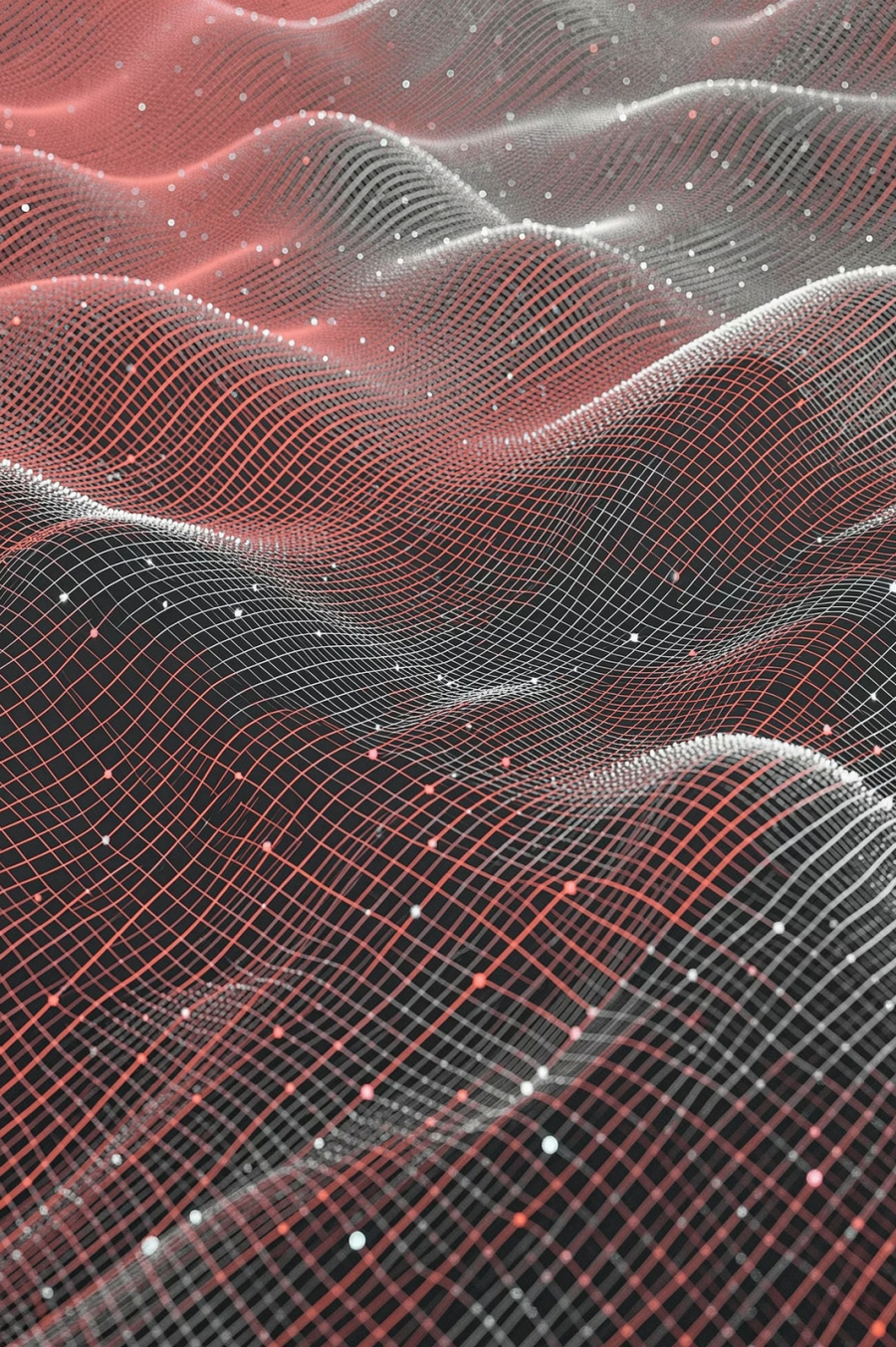


Cart



Checkout

Chronological clickstream data • Dynamic size • Easy insertion/deletion without memory reorganization



Trees: Auto-Complete & Hierarchies

01

Tries (Prefix Trees)

Power Google Ads auto-complete and search suggestions

02

Binary Search Trees

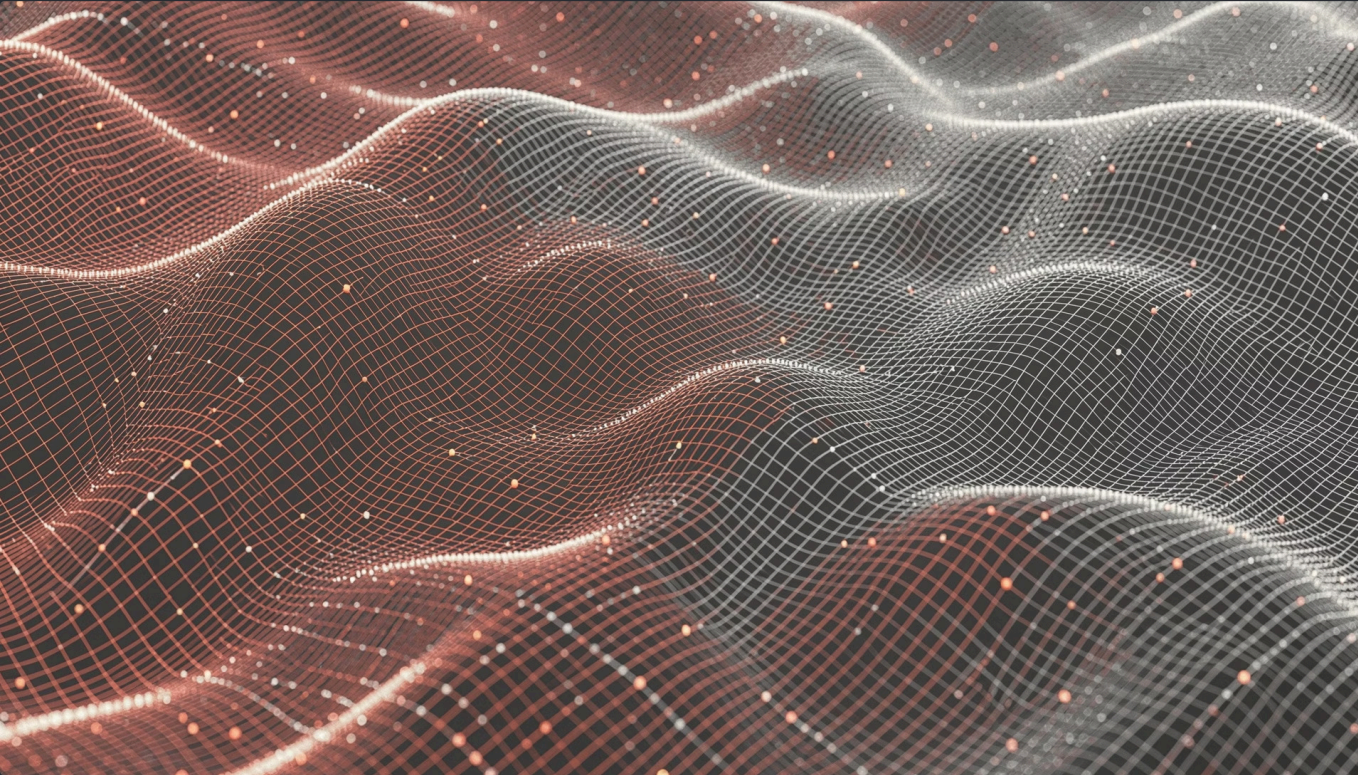
Store campaign hierarchy: Account → Campaign → Ad Group → Ad

03

Efficient Lookup

Faster than linear arrays for sorted data— $O(L)$ predictive text

Graphs: Social Network Analysis



Influencer Mapping & Social Connections

- Map user relationships on social platforms
- Identify key influencers via PageRank logic
- Find shortest path between product and buyer

Code Example: Frequency Capping

```
struct UserView {  
    char userID[20];  
    int viewCount;  
};  
  
int canShowAd(char *id) {  
    int index = hashFunction(id);  
  
    if (hashTable[index]->viewCount < 3) {  
        hashTable[index]->viewCount++;  
        return 1; // Show Ad  
    }  
    return 0; // Block Ad  
}
```

This logic runs millions of times per second on production ad servers

Why C for Marketing Tools?



Performance

No garbage collection pauses—unlike Java/Python



Memory Control

Precise control over every byte for large datasets



Portability

Runs on servers, IoT devices, any hardware



Legacy Libraries

Core marketing analytics built in C/C++

Digital marketing demands data engineering excellence—master data structures to build scalable, high-performance marketing engines